From Heuristics to Graph Neural Networks: A Systematic Experimental Benchmark of Modularity-Based Graph Clustering

Adil Chhabra[1], Kenneth Langedal[1,*], Christian Schulz[1]

[1]Heidelberg University, Heidelberg, Germany
[*]Corresponding author: kenneth.langedal@informatik.uni-heidelberg.de

October 15, 2026

Abstract

Modularity maximization is one of the most widely used objective functions for community detection in networks, balancing intra-community connectivity against a random null model. Over two decades, algorithmic development has split into two largely isolated research streams: classical discrete optimization heuristics operating on topology alone, and deep learning architectures—primarily graph neural networks (GNNs)—that integrate node attributes into continuous modularity relaxations. Because each literature evaluates almost exclusively against baselines from its own paradigm, empirical trade-offs between optimization quality, scalability, and hyperparameter sensitivity remain poorly understood. Here, we present a systematic experimental benchmark evaluating 18 representative modularity-based graph clustering algorithms across seven algorithmic paradigms (greedy agglomerative, multilevel, memetic, streaming, flow/spectral, exact integer linear programming, and neural/GNNs). We evaluate methods across real-world attributed networks, large-scale pure topology benchmarks up to 3.3 billion edges, and synthetic graphs under controlled topological and attribute signal degradation. Across three standardized evaluation protocols isolating out-of-the-box defaults, hardware acceleration, and hyperparameter tuning, our findings reveal clear trade-offs: classical heuristics (notably VieClus and Leiden) dominate pure modularity and scalability without parameter tuning ($\Delta Q \leq 0.002$), while GNN-based methods excel in ground-truth community recovery when informative node attributes exist. However, GNNs carry a heavy hyperparameter tuning tax (ΔQ up to 0.32), lower memory ceilings due to accelerator VRAM constraints, and substantial performance drops when attributes degrade. All algorithms, datasets, and execution harnesses are released within an open-source, reproducible framework.

1 Introduction

Networks provide an expressive foundation for modeling complex relational systems across the natural sciences, social sciences, and engineering [1, 4–6]. In these graphs, vertices represent entities and edges encode pairwise interactions, as observed in social and information networks, communication infrastructures, protein interaction networks, and financial transaction graphs. A ubiquitous structural feature of empirical networks is *community structure*: the partitioning of vertices into cohesive modules characterized by dense internal connectivity and comparatively sparser connectivity across modules.

Uncovering community structure—commonly referred to as *graph clustering* or *community detection*—is a foundational unsupervised learning task with applications spanning functional module detection in biology [5,6], anomaly and fraud detection in financial systems [4], program optimization [2,3], epidemic contact tracing [1], and large-scale web analysis. Most formulations cast community detection as an optimization problem over discrete vertex partitions. Among proposed objectives, *modularity* [7,9] has emerged as the preeminent standard, quantifying the density of intra-community edges relative to their expected value under a degree-preserving null model.

Because exact modularity maximization is strongly NP-hard [8], two decades of research have generated a diverse algorithmic landscape. Crucially, this research has bifurcated into two disjoint paradigms:

1. **Classical combinatorial heuristics**: Operating purely on graph topology, these algorithms range from greedy agglomeration (CNM [10], Louvain [15]) and multilevel local search (Leiden [25]) to memetic evolutionary algorithms (VieClus [22]), streaming clustering (CluStRE [37]), random-walk heuristics (Walktrap [11]), and exact branch-and-cut integer linear programming (Bayan [35]). These methods offer transparent objective tracking, deterministic or bounded stochastic behavior, and high scalability.

2. **Learning-based neural approaches**: Motivated by the ubiquity of rich node attributes (e.g., text, profiles, biological measurements), modern deep learning methods integrate continuous relaxations of modularity into Graph Neural Network (GNN) architectures (DMoN [28], CommDGI [27], GNNS [29], DGCluster [33], MAGI [34], UCoDe [30], LIM [38]). These methods jointly optimize attribute embeddings and structural objectives to capture joint topological-semantic communities.

Despite rapid publication in both domains, the literature remains deeply fragmented. Algorithmic papers introducing GNN-based clusterers predominantly compare against other neural baselines or older heuristic baselines (often decades-old Louvain in Python), while combinatorial algorithm designers rarely evaluate on attributed networks or against modern neural pipelines (see Figure 1). Consequently, foundational questions remain unanswered: *Does deep learning actually improve modularity over modern combinatorial optimizers? How sensitive are neural methods to hyperparameter tuning compared to classical heuristics? What are the true computational and memory bottlenecks when scaling from multi-core CPUs to GPU accelerators?*

Prior surveys have systematically failed to address this intersection. Earlier reviews have focused exclusively on classical combinatorial heuristics [31], restricted their focus to deep learning methods on small citation graphs [46–49], or evaluated exclusively slow Python-based implementations on modest graph sizes [45]. None have jointly evaluated the state-of-the-art across both paradigms under uniform, hardware-monitored conditions.

Contributions

In this work, we present the first large-scale, systematic experimental benchmark that bridges classical and learning-based modularity maximization. Our main contributions are:

- **Comprehensive Algorithmic Coverage**: We evaluate 18 representative clustering algorithms spanning seven optimization paradigms: greedy agglomeration, multilevel

search, memetic evolutionary algorithms, streaming, flow/spectral heuristics, exact integer linear programming, and deep graph neural networks.

- **Unified Three-Tiered Benchmarking Framework**: We introduce three standardized experimental protocols designed to decouple orthogonal research questions:
  - *Protocol A (Sequential Baseline)*: Out-of-the-box author defaults on uniform CPU hardware to evaluate default operational quality and runtime.
  - *Protocol S (Scalability & Accelerators)*: Multi-threaded CPU scaling (including parallel MPI evolutionary search in VieClus) and GPU acceleration to evaluate throughput speedups versus physical VRAM capacity limits.
  - *Protocol B (Hyperparameter Sensitivity & Oracle Upper Bounds)*: Per-instance Bayesian parameter tuning on representative graphs to quantify the *Sensitivity Gap* ($\Delta Q = Q_{\text{tuned}} - Q_{\text{default}}$).
- **Multi-Faceted Empirical Insights**: We evaluate across three complementary benchmark datasets: real-world attributed graphs with ground-truth metadata, large-scale pure topology benchmarks up to 3.3 billion edges (DIMACS challenge suite), and synthetic attributed graphs (ADC-SBM) with systematically degraded topological and attribute signal. Our findings reveal:
  - *Optimization Quality*: Classical heuristics (VieClus and Leiden) achieve the highest modularity on structural graphs, often matching exact solutions.
  - *The Tuning Tax*: Classical heuristics are virtually invariant to hyperparameter tuning ($\Delta Q \leq 0.002$), while GNNs exhibit severe hyperparameter sensitivity (ΔQ up to 0.32), requiring extensive per-instance tuning to approach competitive performance.
  - *The Accelerator VRAM Ceiling*: While GPUs provide up to 78× throughput speedup for GNNs, their lower physical memory capacity (VRAM) imposes a hard scalability ceiling compared to 120 GB host RAM.
  - *Signal Dependency Spectrum*: Under controlled signal degradation, methods display distinct inductive biases: MAGI and DMoN rely heavily on attributes, whereas VieClus, Leiden, GNNS, and DGCluster rely primarily on graph topology.
- **Reproducible Benchmarking Infrastructure**: We open-source our complete experimental harness, providing zero-copy memory-mapped binary CSR graph and feature I/O, cgroup v2 sub-millisecond memory monitoring, standardized solver wrappers, and reproducible trial runners.

2 Preliminary

Graphs and clusterings. An *(attributed) graph* is defined as $G = (V, E, \mathcal{F})$, where $V = \{v_1, v_2, \ldots, v_n\}$ is a set of n vertices, $E \subseteq V \times V$ is the set of undirected edges, and $\mathcal{F} = \{f_1, f_2, \ldots, f_n\}$ denotes node attributes, with each $f_i \in \mathbb{R}^x$ an attribute vector associated with vertex v_i, and x the attribute dimension. We write $n := |V|$ and $m := |E|$ for the number of vertices and edges, respectively. For a vertex $u \in V$, its *neighborhood* is $N(u) := \{v \in V : \{u, v\} \in E\}$, and its *degree* is $deg(u) := |N(u)|$. Let $A \in \{0, 1\}^{n \times n}$ denote the adjacency matrix

of G, where $A_{ij} = 1$ if $\{v_i, v_j\} \in E$ and $A_{ij} = 0$ otherwise. Let $\mathbf{d} \in \mathbb{R}^n$ denote the degree vector with $\mathbf{d}_i = \deg(v_i)$.

A *clustering* C of G is a partition of the vertex set V into k disjoint blocks $V_1, \ldots, V_k$ such that $V_1 \cup \cdots \cup V_k = V$. Classical algorithms operate solely on the graph structure (V, E), while learning-based methods additionally exploit node attributes $\mathcal{F}$. Learning-based approaches often aim to discover communities that are coherent both in terms of graph connectivity and attribute similarity.

Modularity. The *modularity* score of a clustering is the fraction of edges that are within one cluster minus the expected fraction if the edges were distributed at random. Let $L_i := |\{\{u, v\} \in E : u, v \in V_i\}|$ denote the number of edges internal to block V_i. The modularity of C is defined as

$$Q(C) := \sum_{i=1}^{k} \left(\frac{L_i}{m} - \left(\sum_{u \in V_i} \frac{deg(u)}{2m} \right)^2 \right)$$

Equivalently, modularity can be expressed in spectral form. Let $C \in \{0, 1\}^{n \times k}$ denote the (hard) cluster assignment matrix, where $C_{ip} = 1$ if vertex v_i belongs to cluster p and $C_{ip} = 0$ otherwise. Define the *modularity matrix* $B := A - \frac{\mathbf{d}\mathbf{d}^\top}{2m}$. Then modularity can be written as

$$Q(C) = \frac{1}{2m} \operatorname{Tr}(C^\top B C).$$

This spectral formulation serves as the foundation for many continuous and learning-based approaches.

Graph neural networks. Graph neural networks (GNNs) are a class of neural architectures that compute vector representations for vertices in an attributed graph by iteratively aggregating information from local neighborhoods. In the context of community detection, GNNs are used to map vertices into a representation space in which structurally or semantically related vertices are close. Learning-based clustering methods then infer communities indirectly by optimizing objectives such as continuous or differentiable relaxations of modularity, mutual-information maximization, or contrastive criteria defined over these representations or over derived *soft cluster assignments*, which encode the strength of association between nodes and communities.

3 Background and Methods

We review objective functions and algorithmic paradigms for community detection and introduce the taxonomy used in this work.

Many clustering quality measures have been proposed, including *coverage*, *performance* [50], *inter-cluster conductance* [51], *surprise* [52], and *modularity* [9]. Coverage, the fraction of intra-cluster edges, is simple but favors trivial solutions. Modularity mitigates this by comparing observed coverage to its expectation under a null model, and has become one of the most widely used objectives for community detection [54]. It is the primary quality function in large-scale benchmarks such as the DIMACS challenges [53] and is the focus of this work. While Aref et al. [31] report that modularity heuristics rarely attain optimal solutions, Ans'otegui et al. [39] show that the VieClus's modularity maximization heuristic routinely computes optimal

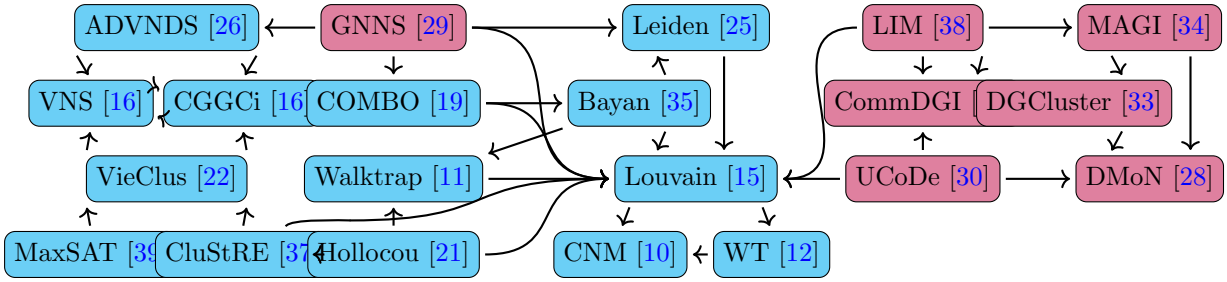


Figure 1: Previous experimental comparisons visualized. Each directed edge indicates that the source algorithm used the destination algorithm in the experimental evaluation. The colors indicate either a learning-based method when red, or a classical optimization algorithm when blue.

solutions on small to medium-sized graphs. Alternative objectives capture complementary notions of community structure: *Weighted Community Cluster* (WCC) maximizes the number of triangles within each community [44], implemented in the Scalable Community Detection algorithm [18], and the *Map Equation*, which measures the expected description length of a random walk [14], used in Infomap [14], Infoflow [24], and the recent learning-based Neuromap [36]. We do not include all the methods based on alternative objective functions in order to maintain our focus on modularity maximization.

Classical modularity-based algorithms use discrete optimization heuristics that consider graph topology only. More recently, learning-based approaches to community detection have been proposed, motivated by the development of GNNs that can model node attributes as feature vectors to capture additional graph information beyond topology alone. These methods typically optimize training objectives that are explicitly aligned with modularity or related structural principles. Compared to classical heuristics, learning-based methods trade exact modularity gains for flexibility, the ability to incorporate node attributes, and increased generalization across graphs. Despite extensive work in both paradigms, existing surveys do not provide a unified experimental comparison of classical and learning-based modularity optimization methods. Prior surveys typically focus on either classical heuristics [31] or learning-based methods [46–49]. Li et al. [45] compare classical and deep learning methods, but limit comparison to Python-based solvers, thus lacking a comprehensive comparison of the state-of-the-art. In addition, algorithmic papers in both paradigms rarely evaluate against methods outside their own class (see Figure 1).

This work addresses this gap by providing a comprehensive experimental comparison of modularity-based community detection algorithms under a unified framework. Here, we provide a taxonomy of the considered methods listed in Table 1. We organize methods by *optimization paradigm*, distinguishing between *classical* combinatorial algorithms and *learning-based* approaches with trainable parameters. Classical methods are further grouped by their dominant optimization mechanism (e.g., greedy, multilevel, or metaheuristic) and by computational setting (e.g., streaming and exact solvers). Learning-based methods are categorized by how modularity is used during training, separating approaches that directly optimize continuous relaxations of modularity from those that use modularity as a guiding signal within broader learning objectives. Within each subgroup, algorithms are listed chronologically.

Table 1: Comparison of benchmarked methods. *Code* indicates implementation availability (×: unavailable; ⋆: available but not ready for use; ⋆⋆: open source with well-defined I/O). *Features* indicate parallelism (P), automatic or user-specified number of clusters (A/K), learning-based methods (L), use of node features (F), and availability in portable Python packages (Py). The bold methods are exact solvers.

Method	Year	Code		Features
CNM [10]	2004	⋆⋆	[60]	. A . . Py
Walktrap [11]	2005	⋆⋆	[61]	. A . . Py
WT [12]	2007	⋆⋆	[61]	. A . . Py
Louvain [15]	2008	⋆⋆	[61]	. A . . Py
CGGCI [17]	2012	×		. A . . .
VNS [16]	2012	×		. A . . .
COMBO [19]	2014	⋆⋆	[62]	. A . . Py
ADVNDS [26]	2017	×		. A . . .
Hollocou [21]	2017	⋆⋆	[63]	. A . . .
VieClus [22]	2018	⋆⋆	[64]	P A . . Py
Leiden [25]	2019	⋆⋆	[61]	. A . . Py
DMoN [28]	2020	⋆	[65]	P K L F .
CommDGI [27]	2020	⋆	[66]	P K L F .
GNNS [29]	2022	⋆	[67]	P A L . .
UCoDe [30]	2023	⋆	[68]	P K L F .
DGCluster [33]	2023	⋆	[69]	P A L F .
MAGI [34]	2024	⋆	[70]	P K L F .
Bayan [35]	2024	⋆⋆	[71]	. A . . .
CluStRE [37]	2025	⋆⋆	[72]	. A . . .
LIM [38]	2025	⋆	[73]	P A L F .
MaxSAT [39]	2025	×		. A . . .

3.1 Classical Modularity Optimization Methods

3.1.1 Greedy agglomeration and multilevel local search

Early approaches to modularity maximization focused on greedy agglomeration, merging communities to directly increase modularity. Subsequent work identified limitations of purely agglomerative strategies, notably unbalanced merges and poor scalability, shifting toward multilevel formulations based on local vertex moves.

Clauset–Newman–Moore (CNM) [10] is a hierarchical agglomerative algorithm that greedily maximizes modularity. Starting from singleton communities (each vertex assigned to its own cluster), it repeatedly merges the pair of communities with the largest modularity gain. The algorithm produces a dendrogram representing the full merge hierarchy and returns the clustering attaining maximum modularity. Its running time is $O(md \log n)$, where d is the dendrogram depth.

Wakita–Tsurumi (WT) [12] refines CNM by addressing its tendency to produce highly unbalanced merges that degrade scalability on large graphs. While preserving the same agglomerative structure, WT modifies merge selection to account for both modularity gain and a community-size ratio, biasing the process toward more balanced agglomerations and improving scalability.

Louvain [15] replaces community-pair merging with greedy local vertex moves. Vertices are iteratively moved to neighboring communities whenever the induced modularity gain is positive. Once no improving move exists, communities are contracted into supernodes and the process is repeated, yielding a multilevel optimization scheme. Each local-moving phase runs in time linear in the number of edges, and the number of multilevel passes is typically small, resulting in near-linear runtime in practice.

Leiden [25] addresses LOUVAIN's tendency to produce disconnected communities by introducing a refinement phase before contraction. This phase splits and restructures communities through probabilistic, non-greedy merges, while guaranteeing that resulting communities are connected and locally optimal.

3.1.2 Non-modularity baselines

We include two widely used non-modularity baselines which rely on random walks to model how information or flow propagates through the network, providing an alternative perspective to modularity optimization.

WalkTrap [11] performs hierarchical clustering based on the intuition that random walks on a graph tend to get "trapped" into densely connected regions corresponding to communities. Each vertex is characterized by the probability distribution over vertices reached by a random walk starting from it, and a structural similarity distance between vertices (and communities) is defined by comparing these distributions. Communities are then obtained by agglomerative merging of the closest pairs under this distance, producing a dendrogram similar to CNM. The algorithm runs in $O(mnd)$ time, where d is the depth of the resulting dendrogram.

Infomap [14] formulates community detection as an information-theoretic problem, seeking a clustering that minimizes the expected description length of a random walk under the map equation. It uses a LOUVAIN inspired greedy multilevel search, augmented with fine- and coarse-tuning steps to escape local minima like LEIDEN.

3.1.3 Metaheuristics and ensemble/hybrid search

Beyond greedy and multilevel heuristics, several methods apply metaheuristic or ensemble-learning strategies to modularity maximization. Representative examples include Variable Neighborhood Search (VNS) [16], which systematically perturbs the current clustering to escape local optima, and ensemble-based approaches such as CGGCi-RG [17], which stabilize modularity optimization by consolidating recurring community structures across multiple runs. These methods combine diversification mechanisms (e.g., randomized perturbations or multiple initializations) with intensification via modularity-based local refinement. While more computationally expensive than purely greedy algorithms, such strategies often achieve higher modularity by exploring a larger search space.

COMBO [19] considers pairs of communities and evaluates full redistributions of vertices between them using Kernighan–Lin–style refinements, instead of relocating individual vertices as in Louvain. By unifying merges, splits, and partial recombinations, COMBO can escape local optima that trap multilevel greedy methods, at the cost of higher computational complexity. The algorithm runs in $O(n^2C)$ time, where C is the number of clusters.

VieClus [22] further extends ensemble-based optimization by adopting a memetic (evolutionary) framework. Rather than refining a single partition, it maintains a population of clusterings and generates new solutions through recombination and mutation. Recombination exploits agreement between high-quality solutions via overlay clustering, where two vertices are grouped together only if they co-occur in both parent solutions, while mutation introduces controlled perturbations to preserve diversity. Each offspring is refined using Louvain or multilevel local search and retained only if it improves modularity. After a set time limit, VieClus returns the best clustering found thus far.

3.1.4 Streaming algorithms

Most classical modularity algorithms assume random access to the full graph, which becomes infeasible for large or dynamically generated networks. Streaming algorithms address this limitation by processing the graph incrementally under strict memory constraints, typically allowing only a single or few passes over the data.

Hollocou [21] is a one-pass edge-streaming algorithm. The algorithm starts from the singleton clustering and processes edges sequentially. For an arriving edge (u, v), Hollocou decides whether to merge the communities of u and v based on their current community volumes. If both volumes are below a fixed threshold, the node belonging to the smaller-volume community joins the other. The authors show that under random stream orderings, this local, volume-based merging rule corresponds to a greedy optimization of modularity in the streaming setting.

CluStRE [37] follows the node-streaming model unlike the edge-streaming model of Hollocou. Here, nodes arrive sequentially together with their neighborhood lists. Starting from singleton clustering, each node is assigned to the cluster among its neighbors that maximizes the modularity gain similar to a Louvain local move. During streaming, a quotient graph is constructed whose nodes represent clusters and whose weighted edges summarize inter- and intra-cluster connectivity, preserving modularity exactly. This quotient graph is optionally refined using VieClus. The graph may be re-streamed to perform further improvements until convergence.

3.1.5 Exact solvers

In contrast to heuristic approaches, exact algorithms provably compute globally optimal modularity clusters. Since modularity maximization is NP-hard [8], these methods rely on exponential-time search augmented with bounding techniques or logical formulations such as weighted partial MaxSAT [39], and are therefore limited to small or medium-sized graphs.

Bayan [35] is an exact branch-and-cut algorithm for modularity maximization. It formulates modularity as an integer program with binary co-membership variables and enforces consistency via triangular constraints on node triples, which branch on whether three nodes must belong to the same community or cannot all be clustered together. Upper bounds are obtained by solving linear programming relaxations, while lower bounds are computed from feasible clusterings produced by COMBO. Branching decisions either contract node triples into supernodes or discourage their co-assignment, and the algorithm terminates once optimality is certified or a prescribed optimality gap is reached.

3.2 Learning-Based Methods

3.2.1 Continuous and differentiable modularity maximization

These approaches replace discrete community assignments with continuous representations, often learned using GNNs, expressing modularity as a continuous or differentiable objective optimized over node representations or soft cluster assignments.

DMoN [28] introduces a differentiable relaxation of modularity and realizes it using a GNN. A GNN followed by a softmax function produces soft cluster assignments, which are optimized using an objective whose main term corresponds to modularity in spectral form. The objective further includes a *collapse regularization* term that prevents trivial solutions in which all nodes are assigned to the same cluster.

GNNS [29] formulates modularity maximization as a continuous optimization process over soft cluster assignments. Starting from random assignments, node assignments are iteratively updated by combining previous assignments with modularity-weighted aggregation of neighboring assignments. Activation functions and normalization steps amplify valid assignments supported by positive modularity contributions. The procedure is repeated for multiple random initializations and parameter choices, and the best converged solution is discretized to obtain the final clustering.

Most learning-based modularity methods, including DMoN, require the number of communities to be specified a priori. **DGCluster** [33] eliminates this requirement by learning node embeddings with a GNN and clustering implicitly through a joint optimization objective. Node embeddings learned by a GNN induce soft pairwise community assignments, which parameterize a differentiable modularity objective. A secondary loss incorporates auxiliary local information, such as partial labels or clusterings from classical heuristics, by aligning embedding similarities with a pairwise similarity matrix. After end-to-end optimization, final communities are obtained by clustering the learned embeddings using BIRCH [59].

While DGCluster removes the need to predefine the number of communities, it relies on jointly optimizing multiple objectives which increases model and training complexity. **LIM** [38] addresses this trade-off with a single-objective formulation that still does not require prior specification of the number of communities. The method first applies Louvain to extract coarse

global communities, which are summarized as structural community centers in a feature space. A graph convolutional network then learns node representations that capture local structural information and node attributes. Soft cluster assignments are computed by measuring similarity between node representations and the structural centers, and a single modularity objective defined over these assignments is optimized to obtain the final clustering.

3.2.2 Contrastive/infomax objectives

These methods do not optimize modularity directly, but instead use it to guide representation learning through alternative objectives.

Community Deep Graph Infomax (CommDGI) [27] learns community-aware node representations by jointly maximizing mutual information between nodes and the graph and between nodes and their communities. Unlike other methods that directly optimize modularity, modularity is used here as an auxiliary objective together with mutual information, via a soft k-means–style clustering layer, to encourage structurally coherent communities.

Unified Community Detection (UCoDe) [30] optimizes a single contrastive objective derived from the modularity matrix, encouraging high similarity within communities and low similarity across communities. This contrastive formulation allows UCoDe to recover non-overlapping communities while naturally supporting overlap, without requiring separate objectives or post-processing. Compared to combined-objective methods such as CommDGI, UCoDe uses a single, interpretable loss and requires minimal hyperparameter tuning.

MAGI [34] establishes a formal connection between modularity maximization and graph contrastive learning by showing that modularity naturally defines positive and negative node pairs via the sign of the modularity matrix. Specifically, node pairs with positive modularity coefficients are treated as positive samples, while pairs with non-positive coefficients are treated as negatives, yielding a contrastive objective equivalent to relaxed modularity maximization. A GNN encoder is trained using a community-aware contrastive loss, with mini-batch modularity matrices constructed via random-walk sampling for scalability.

4 Evaluation Framework

In this section, we describe the benchmarking methodology used to evaluate modularity-based graph clustering algorithms under controlled and reproducible conditions. A central contribution is the integration of 17 clustering algorithms behind a unified benchmarking interface, offering a fair comparison across diverse implementations. In addition to enabling the experiments presented in this survey, it will also serve as an openly available resource for future work on graph clustering, simplifying the evaluation of older methods on new data or machines. This section also includes details on the datasets we used to evaluate the clustering algorithms.

4.1 Benchmarking Interface

As shown in Table 1, over half of the clustering algorithms in the literature lack ready-to-use interfaces for clustering new data. Furthermore, among those with properly documented code, there are wide differences in the input graph format, output formats, and the additional information they report, such as time and memory consumption. This lack of standardization

complicates fair benchmarking, as differences in preprocessing, reporting, and resource accounting can substantially affect observed performance.

To address this problem, we aim to provide an interface with a single common input and output format and consistent time and memory monitoring. The interface exposes a common set of inputs, while allowing solver-specific parameters to be provided when applicable:

$$\textsc{Cluster}(G, T_{\max}, p, k, M_{\max}, \mathcal{F})$$

where G is the input graph, $T_{\max}$ is the wall-clock time limit, p is the number of threads available to the algorithm, k is the number of clusters to compute, $M_{\max}$ is the memory limit in gigabytes, and $\mathcal{F}$ are the graph features. Parameters p, k, and $\mathcal{F}$ are only used by a subset of the solvers. From Table 1, parallel solvers use p, those that compute a fixed number of clusters use k, and any solver that uses graph features can optionally accept $\mathcal{F}$. When $\mathcal{F}$ is not provided, but the solver expects features, a single 1.0 value is assigned to each vertex as the feature. This interface is designed to capture the most common operational requirements across clustering algorithms while enforcing consistent resource limits and output reporting.

Whenever possible, we use the authors' implementations or the most commonly used library implementation. The exact setup for each algorithm is described in our openly available GitHub repository[1]. We make only minimal, deliberate modifications where necessary to ensure consistent timing, memory accounting, and output quality measures based on the following rules:

- Parsing the input and writing the output to a file is not counted in the timing of the algorithm.
- For search-based algorithms that always run for the entire $T_{\max}$ duration, the recorded computation time is defined as the point in time when the best solution is found.
- Learning-based methods that train a neural network as part of the algorithm will break the training loop at $T_{\max}$, and use the current model at that time to produce the final clustering.

4.2 Parameter Settings

Where possible, we use the default configurations for all algorithms. Here, we detail the exact parameters chosen for each algorithm. Algorithms that are not listed do not expose any parameters.

Parameter selection policy. To reflect how algorithms are typically used in practice, we deliberately avoid any dataset-specific or instance-specific parameter tuning. Our goal is to benchmark available implementations as they are, rather than hypothetical optimal configurations that would require extensive tuning effort.

- Walktrap: walk length $= 4$.
- Louvain: resolution $= 1$.
- Hollocou: $v_{\max} = 10\,000$.

[1] https://github.com/ClusterBenchmark/ClusterBenchmark

- VieClus: time limit = 1 hour.
- Leiden: resolution = 1, $\beta = 0.02$, iterations = 2.
- DMoN: 1 hidden layer (64), 1 000 epochs, lr = 0.001.
- CommDGI: 1 hidden layer (512), 1 000 epochs, lr = 0.001.
- GNNS: GNNS100 variant.
- UCoDe: 2 hidden layers (256), 1 000 epochs, lr = 0.001.
- DGCluster: hidden layers (256, 128), 300 epochs, lr = 0.001.
- MAGI: hidden layers (512, 256), 100 epochs, lr = 0.01.
- Bayan: Gurobi ILP solver [42], gap = 10^{-4}, resolution = 1.
- CluStRE: light mode, modularity objective.
- LIM: 1 hidden layer (512), 1 000 epochs, patience = 200, lr = 0.001.

For stochastic algorithms with an explicitly specified seed, we use the run ID as the seed for each of the five runs. While many of these algorithms include multiple configurations, we use either the configuration recommended by the authors or the default configuration provided with the implementation.

4.3 Dataset

We consider three datasets commonly used in the literature. The first set consists of 10 real-world graphs with known community labels; we refer to this dataset as the Ground-Truth graphs. The machine learning algorithms commonly use these graphs, and the details of the instances are shown in Table 2. Next, we consider the instances used in the 10th DIMACS implementation challenge on graph clustering [53]; we call this dataset DIMACS graphs. This dataset is popular among the classical modularity optimization methods, and the details of these graphs are shown in Appendix A. Finally, we use synthetically generated graphs with node features generated using the attributed, degree-corrected stochastic block model (ADC-SBM) [40]. This generator plants k clusters in the graph, and generates random edges via a distribution that is conditional on those clusters. It does so using the two input parameters d and $d_{out} \leq d$, where d is the expected degree for each vertex and d_{out} is the expected number of neighbors for a node in a different cluster than its own. The ADC-SBM generator also produces node features that are correlated with the community structure. It does so using the two input parameters σ for intra-cluster center variance and σ_c for cluster center variance.

5 Experiments

In this section, we present benchmark results obtained using the unified benchmarking interface described in Section 4. We report results on real-world graphs with and without ground-truth labels, as well as on synthetically generated graphs with controlled structural and feature noise. Rather than exhaustively ranking algorithms, these experiments illustrate how different

Table 2: GROUND-TRUTH graphs. The $|C|$ column shows the number of clusters and x the number of features.

Instance	n	m	x	$\lvert C\rvert$
Cora	2 708	5 278	1 433	7
Citeseer	3 327	4 552	3 703	6
Amazon-Photo	7 650	119 081	745	8
Amazon-Computers	13 752	245 861	767	10
Coauthor-CS	18 333	81 894	6 805	15
Pubmed	19 717	44 324	500	3
Coauthor-Physics	34 493	247 962	8 415	5
ogbn-Arxiv	169 343	1 157 799	128	40
Reddit	232 965	57 307 946	602	41
ogbn-Products	2 449 029	61 859 012	100	47

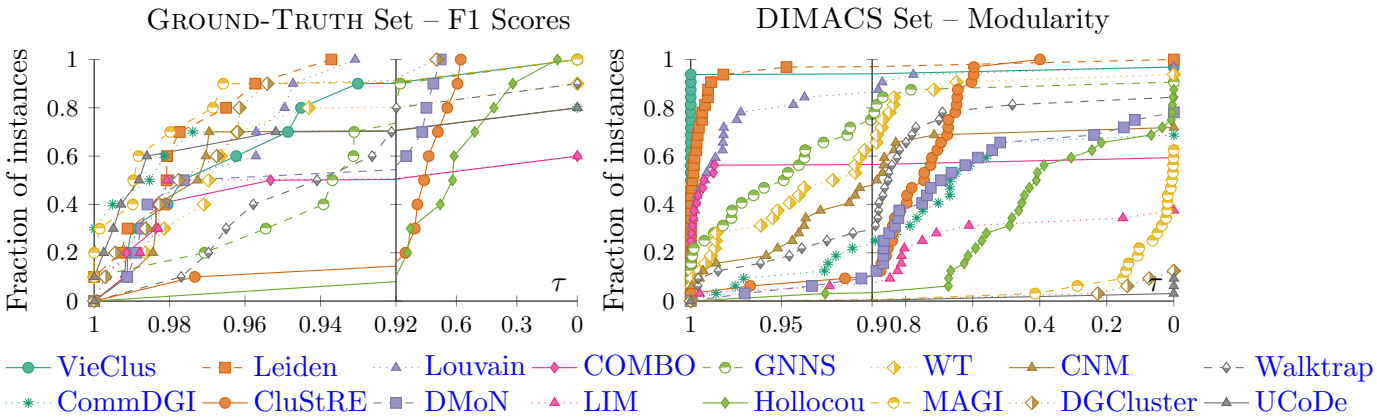


Figure 2: Performance profiles for our first two datasets. We use F1 scores for the GROUND-TRUTH set and modularity for the DIMACS instances. For clarity, we split the x-axis to highlight the differences between the best algorithms near one.

algorithmic design choices behave under a standardized benchmarking setup. To support this, we also provide a dynamic website reporting updated results[2].

5.1 Computational Environment

All algorithms are evaluated on the same machine to eliminate variability due to hardware differences and ensure fair comparison. While some methods could potentially benefit from specialized accelerators such as GPUs, we intentionally restrict all experiments to a shared CPU-based environment to maintain consistency across algorithms. We use a machine with an AMD EPYC 9754 128-core processor with 755 GB of memory, running Ubuntu 24.04.3 with Linux kernel 6.8.0-87. For algorithms with compiled software, we use GCC version 13.3.0, and for Python implementations we use Python version 3.12.3.

To accelerate the overall benchmarking process, we evaluate up to three algorithm instances concurrently on the same machine. This level of concurrency is deliberately conservative relative to the available hardware: the system provides 128 CPU cores and 755 GB of memory, ensuring substantial resource headroom for each run and minimizing contention effects. In particular, we observed that a single process cannot saturate memory bandwidth on this platform, making cross-talk between concurrently evaluated algorithms negligible in practice. Algorithms that

[2] https://clusterbenchmark.github.io/

support parallel execution internally are evaluated with 16 threads. We also record results using a single thread to test scalability. Each algorithm–instance pair is subject to a wall-clock time limit of one hour and a memory limit of 250 GB. Runs exceeding the memory limit are terminated and recorded as *memory limit exceeded* (MLE). Time limits are enforced more leniently, as several learning-based methods train for a fixed number of iterations rather than converging to a stopping criterion; only runs exceeding the time limit by more than 50% are terminated and recorded as *time limit exceeded* (TLE).

For algorithms requiring a prescribed number of clusters, we provide the ground-truth value when available and set $k = 16$ otherwise, following common practice. For stochastic methods, we report the best result over five independent runs. In addition to modularity (Section 2), we evaluate clustering quality using Normalized Mutual Information (NMI) and an edge-restricted pairwise F1 score. The NMI score measures the level of agreement between the predicted clusters and the ground-truth clusters. For more detailed information, refer to Danon et al. [74]. The F1 score is computed over adjacent vertex pairs: an edge (u, v) is a true positive (TP) if u and v are assigned to the same cluster by the algorithm as well as in the ground-truth clusters. Similarly, we define false positives (FP), true negatives (TN), and false negatives (FN). The F1 score is then calculated using the formula: $F1 := \frac{2TP}{2TP+FP+FN}$.

Performance profiles Performance profiles are well-suited for benchmarking multiple algorithms, as they summarize relative performance across instances without requiring direct aggregation of objective values [41]. A performance profile assigns each algorithm a non-decreasing, piecewise-constant function. The y-axis represents the fraction of graphs where the objective value is better than or equal to τ times the best observed objective value from any algorithm. Reading a performance profile, an algorithm performs well if it achieves lower τ values for higher fractions of instances.

5.2 State-of-the-Art Comparisons

Figure 2 shows the main results of the experimental comparison. Across both datasets, VieClus achieves the highest modularity whenever it completes within the imposed resource limits, consistent with prior findings [22,39]. As expected for a memetic search-based method, it exceeds the memory limit on the two largest DIMACS instances.

On the Ground-Truth dataset, many methods achieve similarly high F1 scores. Among them, MAGI achieves the strongest and most stable performance. This highlights a general mismatch between modularity optimization and externally defined communities. For example, the Citeseer ground-truth clustering has a modularity of 0.54, whereas all methods produce clusterings with modularity above 0.54. This discrepancy reflects the fact that citation behavior does not necessarily align with categorical labels, as Figure 1 also indicates.

For the DIMACS graphs, differences between methods are more pronounced, with modularity-focused algorithms clearly outperforming learning-based approaches. Among learning-based methods, performance varies substantially depending on whether the method is designed to operate without node features, with GNNS performing best in this setting.

The streaming methods Hollocou and CluStRE trade solution quality for extreme efficiency. While their F1 scores are lower on the Ground-Truth dataset, CluStRE achieves competitive modularity on DIMACS instances while using orders of magnitude less time and

memory. The exact algorithm Bayan terminates only on the eight smallest instances from the DIMACS dataset and is therefore excluded from the performance profiles. For these graphs, VieClus found the optimal clustering for all of them, Leiden for 7/8, GNNS for 5/8, COMBO for 4/8, and Louvain for 3/8. The remaining algorithms did not find any known optimal solutions.

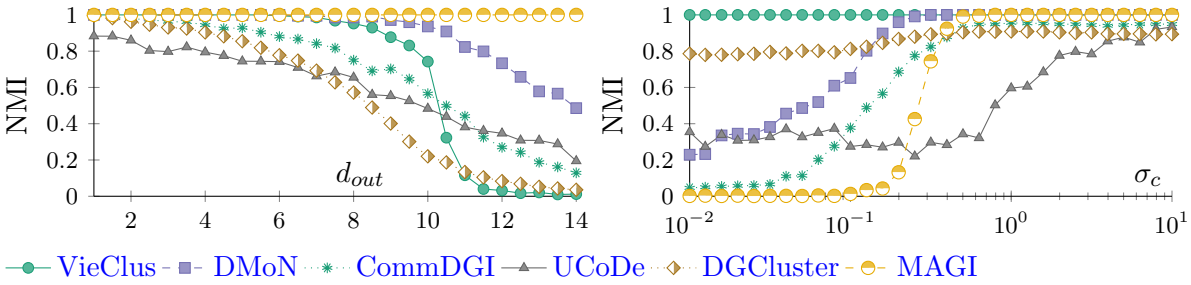


Figure 3: NMI scores on graphs generated with the ADC-SBM graph generator. All graphs have $n = 1\,000$ vertices, $k = 4$ clusters, $d = 20$ average degree, and $\sigma = 1$ intra-cluster center variance. The left plot varies d_{out} while keeping $\sigma_c = 3$, and the right plot varies σ_c while keeping $d_{out} = 2$.

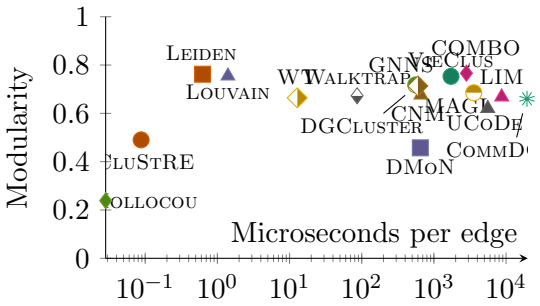


Figure 4: Scatter plot showing the ratio between running time and solution quality, using the geometric mean for both measures on the Ground-Truth graphs. To normalize the running time per graph, we divide it by the number of edges, yielding a time-per-edge measure.

5.3 Quality–Efficiency Tradeoffs

Figure 4 summarizes the tradeoff between clustering quality and execution time using geometric means across instances. Streaming methods occupy the extreme low-latency regime, while greedy modularity methods (Louvain, Leiden) achieve a favorable balance between quality and efficiency. The search-based algorithm VieClus achieves the highest quality, but at a substantially higher computational cost. Similarly, learning-based methods generally attain high solution qualities at substantially higher computational cost.

Peak memory usage varies significantly across methods, and detailed measurements for the Ground-Truth dataset are presented in Appendix B. This appendix also offers insights into the number of training iterations completed by each learning-based algorithm. Although the largest instances often face memory or time constraints, most algorithms complete all training iterations, showing that GPUs or other accelerators would not have improved the results for those smaller graphs. We also evaluate the parallel algorithms sequentially to analyze how additional computing resources affect their performance; detailed measurements are presented in Appendix C.

5.4 Synthetic Graphs

To further evaluate the impact and sensitivity of graph features across clustering algorithms, we include experiments with syntactically generated graphs with vertex features. These experiments illustrate how feature-aware methods respond to controlled degradation of structural and attribute signals, rather than ranking the methods. For reference, we also include VieClus, which performed best among the pure modularity-optimizing methods. We conduct two sets of experiments with the ADC-SBM generator. First, we generate a sequence of graphs with gradually higher d_{out}, causing the graph structure to carry less information about the original k clusters. And second, we generate graphs with strong community structure as we gradually increase the cluster center variance σ_c, leading the graph features to carry less information about the clusters.

Figure 3 summarizes the results of the two ADC-SBM experiments. When the feature signal is strong, MAGI and DMoN achieve the highest NMI scores, and beyond $d_{out} \approx 11$, all feature-aware methods outperform VieClus. The results also highlight clear differences in how methods balance structural and feature information. MAGI relies primarily on node features, as evidenced by its strong performance under structural noise and its sensitivity to feature degradation. In contrast, DMoN remains robust in both settings, indicating a more balanced use of topology and attributes. CommDGI exhibits similar trends but performs consistently worse than DMoN. DGCluster shows the opposite bias to MAGI, favoring graph structure over features, while UCoDe displays a similar but less pronounced tendency. We exclude LIM from this figure, as it consistently collapsed to a single cluster, yielding zero NMI across all configurations.

6 Conclusion

We surveyed 17 modularity-based graph clustering algorithms, covering classical heuristics, parallel methods, and recent learning-based approaches. When modularity maximization is the sole objective, the results are clear: VieClus consistently achieves the highest modularity values when sufficient memory is available. Leiden, an improved variant of Louvain, closely follows in solution quality and offers substantially faster runtimes. This trade-off reflects their optimization strategies: VieClus relies on time-budgeted search, while Leiden uses a greedy refinement scheme.

Beyond modularity alone, other algorithms exhibit their own unique strengths. Streaming methods such as CluStRE offer extremely low runtimes and memory footprints, making them well-suited for large-scale or resource-constrained settings. Learning-based approaches, including GNNS, demonstrate that neural methods are approaching the performance of hand-crafted heuristics. However, they still require substantially more memory and do not yet achieve competitive modularity scores. At the same time, there is growing interest in expanding modularity optimization with auxiliary objectives to recover ground-truth communities better. In this setting, feature-aware methods can be advantageous: MAGI achieves the strongest ground-truth recovery in our experiments, while DMoN remains effective when community structure is encoded between topology and node attributes.

To support future work in this area, we introduce a unified benchmarking framework that enables direct, reproducible comparisons across a wide range of modularity-based clustering

algorithms. We hope that this framework lowers the barrier for evaluating new methods against established baselines and encourages more systematic experimental studies of graph clustering algorithms.

Acknowledgements

We acknowledge support by DFG grant SCHU 2567/5-1.

Author Contributions

A.C., K.L., and C.S. conceived the study. K.L. and A.C. developed the benchmark framework, performed the experiments, and analyzed the data. K.L., A.C., and C.S. wrote the manuscript.

Competing Interests

The authors declare no competing interests.

Data and Code Availability

The complete benchmarking harness, solver wrappers, and datasets are open-source and publicly available at https://github.com/TODO/ClusterBenchmark.

References

[1] Newman, M. E. J. Properties of highly clustered networks. *Phys. Rev. E* **68**, 026121 (2003). URL https://link.aps.org/doi/10.1103/PhysRevE.68.026121.

[2] McFarling, S. Program optimization for instruction caches. In Emer, J. S. & Hennessy, J. L. (eds.) *ASPLOS-III Proceedings - Third International Conference on Architectural Support for Programming Languages and Operating Systems, Boston, Massachusetts, USA, April 3-6, 1989*, 183–191 (ACM Press, 1989). URL https://doi.org/10.1145/70082.68200.

[3] Demme, J. & Sethumadhavan, S. Approximate graph clustering for program characterization. *ACM Trans. Archit. Code Optim.* **8**, 21:1–21:21 (2012). URL https://doi.org/10.1145/2086696.2086700.

[4] Akoglu, L., Tong, H. & Koutra, D. Graph based anomaly detection and description: a survey. *Data Min. Knowl. Discov.* **29**, 626–688 (2015). URL https://doi.org/10.1007/s10618-014-0365-y.

[5] Xu, Y., Olman, V. & Xu, D. Clustering gene expression data using a graph-theoretic approach: an application of minimum spanning trees. *Bioinform.* **18**, 536–545 (2002). URL https://doi.org/10.1093/bioinformatics/18.4.536.

[6] Cai, B., Wang, H., Zheng, H. & Wang, H. Detection of functional modules from protein interaction networks with an enhanced random walk based algorithm. *Int. J. Comput. Biol. Drug Des.* **4**, 290–306 (2011). URL https://doi.org/10.1504/IJCBDD.2011.041416.

[7] Newman, M. E. J. Modularity and community structure in networks. *Proceedings of the National Academy of Sciences* **103**, 8577–8582 (2006). URL https://www.pnas.org/doi/abs/10.1073/pnas.0601602103. https://www.pnas.org/doi/pdf/10.1073/pnas.0601602103.

[8] Brandes, U. *et al.* On modularity clustering. *IEEE Trans. Knowl. Data Eng.* **20**, 172–188 (2008). URL https://doi.org/10.1109/TKDE.2007.190689.

[9] Girvan, M. & Newman, M. E. J. Community structure in social and biological networks. *Proceedings of the National Academy of Sciences* **99**, 7821–7826 (2002). URL https://www.pnas.org/doi/abs/10.1073/pnas.122653799. https://www.pnas.org/doi/pdf/10.1073/pnas.122653799.

[10] Clauset, A., Newman, M. E. J. & Moore, C. Finding community structure in very large networks. *Physical Review E* **70**, 066111 (2004).

[11] Pons, P. & Latapy, M. Computing communities in large networks using random walks. In *Proceedings of the 20th International Symposium on Computer and Information Sciences (ISCIS 2005)*, 284–293 (Springer Berlin Heidelberg, 2005).

[12] Wakita, K. & Tsurumi, T. Finding community structure in mega-scale social networks: [extended abstract]. In *Proceedings of the 16th International Conference on World Wide Web (WWW 2007)*, 1275–1276 (Association for Computing Machinery, 2007).

[13] Leicht, E. A. & Newman, M. E. J. Community structure in directed networks. *Physical Review Letters* **100**, 118703 (2008).

[14] Rosvall, M. & Bergstrom, C. T. Maps of random walks on complex networks reveal community structure. *Proceedings of the National Academy of Sciences* **105**, 1118–1123 (2008).

[15] Blondel, V. D., Guillaume, J.-L., Lambiotte, R. & Lefebvre, E. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment* **2008**, P10008 (2008).

[16] Aloise, D. *et al.* Modularity maximization in networks by variable neighborhood search. In *Graph Partitioning and Graph Clustering*, vol. 588 of *Contemporary Mathematics*, 113–127 (American Mathematical Society, 2012).

[17] Ovelgönne, M. & Geyer-Schulz, A. An ensemble learning strategy for graph clustering. In *Graph Partitioning and Graph Clustering*, vol. 588 of *Contemporary Mathematics*, 187–205 (American Mathematical Society, 2012).

[18] Prat-Pérez, A., Dominguez-Sal, D. & Larriba-Pey, J.-L. High quality, scalable and parallel community detection for large real graphs. In *Proceedings of the 23rd International Conference on World Wide Web (WWW 2014)*, 225–236 (Association for Computing Machinery, 2014).

[19] Sobolevsky, S., Campari, R., Belyi, A. & Ratti, C. General optimization technique for high-quality community detection in complex networks. *Physical Review E* **90**, 012811 (2014).

[20] Zhang, P. & Moore, C. Scalable detection of statistically significant communities and hierarchies, using message passing for modularity. *Proceedings of the National Academy of Sciences* **111**, 18144–18149 (2014).

[21] Hollocou, A., Maudet, J., Bonald, T. & Lelarge, M. A streaming algorithm for graph clustering. arXiv preprint (2017).

[22] Biedermann, S., Henzinger, M., Schulz, C. & Schuster, B. Memetic graph clustering. In *Proceedings of the 17th International Symposium on Experimental Algorithms (SEA 2018)*, 3:1–3:15 (Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018).

[23] Gao, H. & Huang, H. Deep attributed network embedding. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI 2018)*, 3364–3370 (International Joint Conferences on Artificial Intelligence Organization, 2018).

[24] Fung, P. K. InfoFlow: A distributed algorithm to detect communities according to the map equation. *Big Data and Cognitive Computing* **3**, 42 (2019).

[25] Traag, V. A., Waltman, L. & van Eck, N. J. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports* **9**, 5233 (2019).

[26] Džamić, D., Aloise, D. & Mladenović, N. Ascent-descent variable neighborhood decomposition search for community detection by modularity maximization. *Annals of Operations Research* **272** (2019).

[27] Zhang, T. *et al.* CommDGI: Community detection oriented deep graph infomax. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management (CIKM 2020)*, 1843–1852 (Association for Computing Machinery, 2020).

[28] Tsitsulin, A., Palowitch, J., Perozzi, B. & Müller, E. Graph clustering with graph neural networks. *Journal of Machine Learning Research* **24**, 1–21 (2023). URL http://jmlr.org/papers/v24/20-998.html.

[29] Sobolevsky, S. & Belyi, A. Graph neural network inspired algorithm for unsupervised network community detection. *Applied Network Science* **7**, 63 (2022).

[30] Moradan, A., Draganov, A., Mottin, D. & Assent, I. UCoDe: Unified community detection with graph convolutional networks. *Machine Learning* **112**, 5057–5080 (2023).

[31] Aref, S., Mostajabdaveh, M. & Chheda, H. Heuristic modularity maximization algorithms for community detection rarely return an optimal partition or anything similar. In *Proceedings of the 23th International Conference on Computational Science (ICCS 2023)*, 612–626 (Springer Nature Switzerland, 2023).

[32] Madi, S. A. & Pirrò, G. Node-centric community deception based on safeness. *IEEE Transactions on Computational Social Systems* **11**, 2955–2965 (2024).

[33] Bhowmick, A., Kosan, M., Huang, Z., Singh, A. & Medya, S. DGCLUSTER: A neural framework for attributed graph clustering via modularity maximization. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence (AAAI 2024)*, 11069–11077 (AAAI Press, Washington, DC, USA, 2024).

[34] Liu, Y. *et al.* Revisiting modularity maximization for graph clustering: A contrastive learning perspective. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2024)*, 1968–1979 (Association for Computing Machinery, New York, NY, United States, 2024).

[35] Aref, S., Mostajabdaveh, M. & Chheda, H. Bayan algorithm: Detecting communities in networks through exact and approximate optimization of modularity. *Physical Review E* **110**, 044315 (2024).

[36] Blöcker, C., Tan, C. & Scholtes, I. The map equation goes neural: Mapping network flows with graph neural networks. In *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS 2024)*, 17428–17456 (Curran Associates, Inc., 2024).

[37] Chhabra, A., Dorian Peretz, S. & Schulz, C. CluStRE: Streaming graph clustering with multi-stage refinement. In *Proceedings of the 23rd International Symposium on Experimental Algorithms (SEA 2025)*, 11:1–11:20 (Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025).

[38] Wang, H. *et al.* Simple yet effective heuristic community detection with graph convolution network. *Scientific Reports* **15**, 39249 (2025).

[39] Ansótegui, C., Ramaswamy, V. P., Szeider, S. & Xia, H. Uncovering and verifying optimal community structure in complex networks: A MaxSAT approach. In *Proceedings of the 25th International Conference on Computational Science (ICCS 2025)*, 35–49 (Springer Nature Switzerland, 2025).

[40] Tsitsulin, A., Rozemberczki, B., Palowitch, J. & Perozzi, B. Synthetic graph generation to benchmark graph learning. arXiv preprint (2022).

[41] Dolan, E. D. & Moré, J. J. Benchmarking optimization software with performance profiles. *Mathematical Programming* **91**, 201–213 (2002).

[42] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual (2026). URL https://www.gurobi.com.

[43] Fortunato, S. & Barthelemy, M. Resolution limit in community detection. *Proceedings of the national academy of sciences* **104**, 36–41 (2007).

[44] Prat-Pérez, A., Dominguez-Sal, D., Brunat, J. M. & Larriba-Pey, J.-L. Shaping communities out of triangles. In *Proceedings of the 21st ACM international conference on Information and knowledge management*, 1677–1681 (Association for Computing Machinery, 2012).

[45] Li, J. *et al.* A comprehensive review of community detection in graphs. *Neurocomputing* **600**, 128169 (2024). URL https://doi.org/10.1016/j.neucom.2024.128169.

[46] Jin, D. *et al.* A survey of community detection approaches: From statistical modeling to deep learning. *IEEE Trans. Knowl. Data Eng.* **35**, 1149–1170 (2023). URL https://doi.org/10.1109/TKDE.2021.3104155.

[47] Liu, F. *et al.* Deep learning for community detection: Progress, challenges and opportunities. In Bessiere, C. (ed.) *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020*, 4981–4987 (ijcai.org, 2020). URL https://doi.org/10.24963/ijcai.2020/693.

[48] Messias, G. H., Gonçalves Botelho, T., Iasulaitis, S., Coelho Naldi, M. & Demétrius Baria Valejo, A. A systematic review of graph neural network for node clustering in homogeneous attributed networks. *IEEE Access* **13**, 215265–215285 (2025).

[49] Liang, K. *et al.* A survey of knowledge graph reasoning on graph types: Static, dynamic, and multi-modal. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **46**, 9456–9478 (2024).

[50] Gaertler, M. Clustering. In Brandes, U. & Erlebach, T. (eds.) *Network Analysis: Methodological Foundations [outcome of a Dagstuhl seminar, 13-16 April 2004]*, vol. 3418 of *Lecture Notes in Computer Science*, 178–215 (Springer, 2004). URL https://doi.org/10.1007/978-3-540-31955-9_8.

[51] Kannan, R., Vempala, S. S. & Vetta, A. On clusterings: Good, bad and spectral. *J. ACM* **51**, 497–515 (2004). URL https://doi.org/10.1145/990308.990313.

[52] Arnau, V., Mars, S. & Marín, I. Iterative Cluster Analysis of Protein Interaction Data. *Bioinformatics* **21**, 364–378 (2004).

[53] Bader, D. A., Meyerhenke, H., Sanders, P. & Wagner, D. (eds.). *Graph Partitioning and Graph Clustering, 10th DIMACS Implementation Challenge Workshop, Georgia Institute of Technology, Atlanta, GA, USA, February 13-14, 2012. Proceedings*, vol. 588 of *Contemporary Mathematics* (American Mathematical Society, 2013). URL https://doi.org/10.1090/conm/588.

[54] Li, J. *et al.* A Comprehensive Review of Community Detection in Graphs. *Neurocomputing* **600**, 128169 (2024).

[55] Rosvall, M., Axelsson, D. & Bergstrom, C. T. The Map Equation. *The European Physical J. Special Topics* **178**, 13–23 (2009).

[56] Peel, L., Larremore, D. B. & Clauset, A. The ground truth about metadata and community detection in networks. *CoRR* **abs/1608.05878** (2016). URL http://arxiv.org/abs/1608.05878. 1608.05878.

[57] Wani, A. A. Comprehensive analysis of clustering algorithms: exploring limitations and innovative solutions. *PeerJ Comput. Sci.* **10**, e2286 (2024). URL https://doi.org/10.7717/peerj-cs.2286.

[58] Zhang, T., Ramakrishnan, R. & Livny, M. Birch: an efficient data clustering method for very large databases. *SIGMOD Rec.* **25**, 103–114 (1996). URL https://doi.org/10.1145/235968.233324.

[59] Zhang, T., Ramakrishnan, R. & Livny, M. Birch: an efficient data clustering method for very large databases. In *Proceedings of the 1996 ACM SIGMOD International Conference*

on Management of Data, SIGMOD '96, 103–114 (Association for Computing Machinery, New York, NY, USA, 1996). URL https://doi.org/10.1145/233269.233324.

[60] Rossetti, G., Milli, L. & Cazabet, R. CDLIB: a python library to extract, compare and evaluate communities from complex networks. *Applied Network Science* **4**, 2364–8228 (2019).

[61] Csardi, G. & Nepusz, T. The igraph software package for complex network research. *InterJournal* **Complex Systems**, 1695 (2006). URL https://igraph.org.

[62] Kats, P. & Belyi, A. COMBO repository. https://github.com/Casyfill/pyCombo.git (2026).

[63] Hollocou, A. Hollocou repository. https://github.com/ahollocou/graph-streaming.git (2026).

[64] Schulz, C. & Seemaier, D. VieClus repository. https://github.com/VieClus/VieClus.git (2026).

[65] Tsitsulin, A., Palowitch, J., Perozzi, B. & Müller, E. DMoN repository. https://github.com/google-research/google-research.git (2026).

[66] Zhang, T. *et al.* CommDGI repository. https://github.com/FDUDSDE/CommDGI.git (2026).

[67] Belyi, A. GNNS repository. https://github.com/Alexander-Belyi/GNNS.git (2026).

[68] Moradan, A., Draganov, A., Mottin, D. & Assent, I. UCoDe repository. https://github.com/AU-DIS/UCODE.git (2026).

[69] Kosan, M. DGCluster repository. https://github.com/pyrobits/DGCluster.git (2026).

[70] Li, J. MAGI repository. https://github.com/EdisonLeeeee/MAGI.git (2026).

[71] Mostajabdaveh, M. & Aref, S. Bayan repository. https://github.com/saref/bayan.git (2026).

[72] Chhabra, A. CluStRE repository. https://github.com/KaHIP/CluStRE.git (2026).

[73] Wang, H. LIM repository. https://github.com/wuanghoong/Less-is-More.git (2026).

[74] Danon, L., Diaz-Guilera, A., Duch, J. & Arenas, A. Comparing community structure identification. *Journal of Statistical Mechanics: Theory and Experiment* **2005**, 2364–8228 (2005).

A Benchmark Instances

Table 3: Graph from the 10th DIMACS implementation challange. There are no features or ground truth clusters for these graphs.

Instance	n	m
karate	34	78
chesapeake	39	170
dolphins	62	159
lesmis	77	254
polbooks	105	441
adjnoun	112	425
football	115	613
jazz	198	2 742
celegansneural	297	2 148
celegans-metabolic	453	2 025
email	1 133	5 451
polblogs	1 490	16 715
netscience	1 589	2 742
power	4 941	6 594
hep-th	8 361	15 751
PGPgiantcompo	10 680	24 316
astro-ph	16 706	121 251
cond-mat	16 726	47 594
as-22july06	22 963	48 436
cond-mat-2003	31 163	120 029
cond-mat-2005	40 421	175 691
preferentialAttachment	100 000	499 985
smallworld	100 000	499 998
G-n-pin-pout	100 000	501 198
caidaRouterLevel	192 244	609 066
cnr-2000	325 557	2 738 969
eu-2005	862 664	16 138 468
in-2004	1 382 908	13 591 473
road-central	14 081 816	16 933 413
uk-2002	18 520 486	261 787 258
road-usa	23 947 347	28 854 312
uk-2007-05	105 896 555	3 301 876 564

B Memory Usage

Table 4: Memory consumption of the evaluated algorithms on the Ground-Truth dataset. Values are reported in human-readable units. Entries marked TLE indicate runs exceeding the one-hour time limit, while MLE denotes runs exceeding the 250 GB memory limit.

Method	Cora	Citeseer	Amz-Photo	Amz-Comp	CoAuth-CS	Pubmed	CoAuth-Phys	ogbn-Arx
VieClus	22.53 MB	22.53 MB	64.91 MB	130.24 MB	58.7 MB	44.94 MB	134.75 MB	497.74 M
Leiden	20.48 MB	20.48 MB	38.91 MB	59.39 MB	34.82 MB	26.62 MB	61.44 MB	211.36 M
Louvain	20.48 MB	20.48 MB	38.91 MB	59.39 MB	32.77 MB	28.67 MB	61.44 MB	210.32 M
COMBO	255.58 MB	314.59 MB	1.09 GB	3.17 GB	5.42 GB	TLE	TLE	MI
GNNS	1.35 GB	1.75 GB	5.49 GB	8.1 GB	8.28 GB	8.31 GB	8.11 GB	6.09 G
WT	20.48 MB	20.48 MB	38.91 MB	59.39 MB	34.82 MB	28.67 MB	61.44 MB	211.36 M
CNM	192.51 MB	179.96 MB	253.07 MB	372.06 MB	234.14 MB	210.71 MB	398.65 MB	TI
Walktrap	27.32 MB	20.48 MB	418.48 MB	1.47 GB	935.16 MB	488.84 MB	4.36 GB	99.06 G
CommDGI	735.03 MB	857.03 MB	2.05 GB	5.34 GB	9.54 GB	9.82 GB	31.09 GB	MI
CluStRE	6.14 MB	6.14 MB	6.14 MB	6.14 MB	6.14 MB	6.14 MB	6.14 MB	8.19 M
DMoN	664.01 MB	863.31 MB	778 MB	987.44 MB	3.5 GB	809.3 MB	7.39 GB	735.92 M
LIM	674.08 MB	741.17 MB	1.92 GB	4.67 GB	7.45 GB	TLE	TLE	MI
Hollocou	2.05 MB	2.05 MB	4.1 MB	6.14 MB	4.1 MB	4.1 MB	8.19 MB	32.39 M
MAGI	807.82 MB	992.96 MB	1.55 GB	2.56 GB	6.15 GB	2.22 GB	15.66 GB	TI
DGCluster	548.86 MB	917.5 MB	879.83 MB	1.39 GB	5.84 GB	1.07 GB	13.2 GB	624.64 M
UCoDe	576.89 MB	771.56 MB	1.7 GB	4.32 GB	7.69 GB	6.94 GB	24.23 GB	MI

Table 5: Number of finished training iterations of the evaluated algorithms on the Ground-Truth dataset. Entries marked TLE indicate runs exceeding the one-hour time limit, while MLE denotes runs exceeding the 250 GB memory limit.

Method	Cora	Citeseer	Amz-Photo	Amz-Comp	CoAuth-CS	Pubmed	CoAuth-Phys	ogbn-Ar
CommDGI	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1 000	628/1 000	708/1 000	224/1 000	M
DMoN	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1
LIM	1 000/1 000	1 000/1 000	1 000/1 000	874/1 000	552/1 000	TLE	TLE	M
MAGI	100/100	100/100	100/100	100/100	100/100	100/100	31/100	T
DGCluster	300/300	300/300	300/300	300/300	300/300	300/300	300/300	300/
UCoDe	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1 000	450/1 000	M

C Parallel-Sequential Comparison

Table 6: Parallel and sequential running times for six instances from the Ground-Truth dataset. Only the parallel methods are included in the table. The t_1 column gives the sequential running time in seconds, while the t_{16} column displays the running time with 16 threads.

Method	Cora		Citeseer		Amz-Photo		Amz-Comp		CoAuth-CS		CoAut
	t_1	t_{16}	t_1	t_{16}	t_1	t_{16}	t_1	t_{16}	t_1	t_{16}	t_1
VieClus	276.55	83.31	8.29	5.19	93.29	16.25	2 804.96	2 476.70	2 811.52	2 897.85	3 511.06
GNNS	36.40	9.41	48.44	11.90	231.92	43.46	592.24	115.08	668.79	162.44	1 906.75
CommDGI	936.72	140.64	2 046.85	288.34	3 606.69	1 284.26	3 616.27	3 416.34	3 613.43	3 602.34	3 664.35
DMoN	55.60	42.19	100.31	50.01	312.63	184.55	658.25	352.90	868.59	212.61	2 060.92
LIM	811.90	97.63	1 218.18	185.00	3 605.01	618.07	3 614.08	1 909.42	3 633.77	2 130.90	TLE
MAGI	119.29	56.88	215.38	66.44	1 445.31	535.17	3 605.81	1 572.78	3 635.58	3 414.73	3 691.43
DGCluster	44.78	17.40	91.01	23.75	301.28	98.98	594.48	193.56	833.56	140.19	1 961.84
UCoDe	223.57	32.04	460.72	62.13	1 305.44	298.82	3 603.07	1 136.63	3 607.82	2 287.33	3 625.18